



Multi-resolution Analysis Based Time-Domain Audio Source Separation with Optimized U-NET Model

Baishakhi Dutta¹ · Chandrakant Gaikwad¹

Received: 13 March 2024 / Revised: 15 November 2024 / Accepted: 18 November 2024 /

Published online: 4 December 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024, corrected publication 2024

Abstract

Audio source separation (ASS) is a technique well-known for extracting the individual signal of underlying sound sources from the signal mixture, which is a hectic challenge due to the presence of multi-channel signals. Though there are numerous features associated with the existing separation models, time-domain signals and phase information are not considered in the conventional techniques. On the other hand, separation models working with DNN are not so efficient with sampling and the separation accuracy of the signal. To overcome these drawbacks, the optimized U-Net model with the Hybrid Wolf Optimization (HWO-UNet) is proposed in this research, which aids in separating the audio source with better accuracy. Utilizing the Hybrid optimized U-Net model with skip connection integrates the spectrogram features and the statistical features effectively for eliminating the loss of signal features and assists in the effective separation of the audio signal as well as the noise signal. The proposed method adopts the U-NET-based architecture with optimized intermediate spectrogram transformation blocks utilizing the adaptive tuning behavior of Deep CNN. Further, the U-Net acts as an encoder–decoder architecture that effectively works as an oversampling technique that mitigates the class imbalance problem and the anti-aliasing technique enhances the efficiency. The performance of the HWO-UNet ASS method in terms of Root Mean Square Error (RMSE), and Mean Square Error (MSE) is 1.242, and 0.912, respectively for the MUSDB18 whereas the maximal Signal-to-Noise Ratio (SNR) is reported as 16.51 dB for UrbanSound8k dataset.

✉ Baishakhi Dutta
baishakhidutta@gmail.com

Chandrakant Gaikwad
cjgaikwad@gmail.com

¹ Department of Electronics and Telecommunication, Ramrao Adik Institute of Technology, DYPU, Nerul, India

Keywords Audio source separation · Hybrid wolf optimization · U-Net · Spectrogram · Deep convolutional neural network

1 Introduction

ASS [46] is a method through which existing sources of an observable audio mixture are extracted [41]. To unmix the noises, from the respective speakers and multichannel mixture has become the most tremendous issue [51]. The separation process can be of two types, such as blind separation as well as informed separation. The blind separation [25] holds only the mixture signals, whereas the informed separation [25] carries additional information about the sources [43, 52]. The ASS problems occur, when certain features, like source characteristics, mixing process characteristics, and dimensionality are unstable and mathematically ill-posed [51]. The principle of ASS is utilized in several microphone devices, such as hearing aids, voice assistants, smartphones, automatic music transcription, and music remixing [9, 13]. Most of the researchers worked with modern applications and the recent source separation research can be classified into speech and Music source separation (MSS) [32], where the former separation method separated the interfering noise and the speech signal identified in the mixture with speech while the latter separation method extracts the sound of each musical instrument from a mixture [16, 32]. Music signals have unique characteristics that obviously differentiate them from other audio signals including the speech and environmental sounds incurred from the reverberant space [3]. Further, understanding these unique characteristics of the music signal is essential as these characteristics of the music signal are exploited on designing the MSS techniques. Usually, the MSS methods involve more than two sources, which creates an under-determined source separation problem [22], which can be resolved when working with neural networks [16].

In recent decades, advances in a variety of study domains have been made possible by Deep Learning (DL), a subset of Machine learning. Image identification, signal processing, and communication issues have been solved using this DL technique. The development of DL techniques also signifies a breakthrough in signal separation [37]. U-Net is a unique deep architecture based on CNN that has numerous convolutional layers. U-Net was first suggested for biomedical image segmentation, in which the UNet architecture has shown to be very successful at precisely segmenting the anatomical features and lesions in medical images because of its encoder-decoder design and skip connections and since the world of image and video processing has seen a major surge in its popularity. Similar to CNN, there are numerous variations of U-Net that can even process the audio directly in the time domain such as Tiny Recurrent U-Net, attention Wave-U-Net [27], and Wave-U-Net [44, 48] adopted for speech enhancement. In addition, an image CNN predicts the class of the entire image, which sets it apart from an image U-Net. In contrast, every pixel in an image is classified using the U-Net image. Further, U-Net attained the new state-of-the-art results in ASS applications [39].

Numerous techniques are formulated for audio source separation to eliminate the impact of noise and separate the source signals effectively [50]. However, the conventional feature extraction techniques have certain limitations due to the outliers and distribution of anisotropic variance that affect the separation accuracy [37]. In addition, the non-clustered algorithms [47, 54] form a spectrum and take advantage of the time–frequency domain in terms of phase and amplitude on which only a limited number of components remain active making the method inefficient at major cases. Furthermore, the SCM kernel-based model disjointly represents and estimates the temporal and level differences among the array channels. Nevertheless, these full-rank spatial models were constrained, by their high sensitivity computing cost and parameter initialization [50]. In [28], the author used a beamspace data model that accounts for intrinsic phase-difference information contained in the recording but fails to work with the data-driven sources. Methods belonging to the first category include Independent Vector Analysis (ISA) [10], Non-negative Matrix Factorization (NMF) [24], Bayesian modeling [38] Independent Component Analysis [49], and Robust Principal Component Analysis (PCA) [35] requires manual interpretation as well as they consume huge annotation time. Moreover, a substantial number of isolated source signals is needed for training the deep spectrum models that are prevalent in use. It is practically difficult to create such supervised training data since the majority of natural audio occurrences are primarily captured in mixture signals [14]. Hence DNNs have become an alternative to existing ASS methods. With the recent development of DNN, ASS performance in supervised contexts has improved [41]. When working with source separation in the magnitude or power spectrogram domain, most DNN algorithms ignore phase information throughout the separation process, which could produce less-than-ideal results [44].

To overcome the above limitations of the existing techniques, the research focuses on developing the optimized U-Net model to separate audio sources. Figure 1 depicts the schematic representation of the proposed framework for the ASS. Initially, audio sources are obtained from the LibriSpeech, MUSDB18 whereas the noise sources are obtained from the UrbanSound8k dataset. The signal is fed into the preprocessor stage, which assists in boosting the model's accuracy and efficiency. Further, the combined spectrogram and statistical features are utilized that form input to the classifier. Features are fed to the U-Net model that is trained applying the HWO algorithm, where the signal is separated as audio as well as the noise signal based on the backpropagation feature of Deep CNN.

- **Hybrid Wolf Optimization:** Utilizing the Hybrid wolf optimization, which incorporates the hunt-search characteristics of the wolves as well as the predator–prey characteristics of the hawks enhances the U-Net model's training. In HWO optimization, the prey's location is determined and the predator's adaptive hunting strategies are adopted to accelerate the convergence speed and to avoid the model falling into the local optimal solution. The training of U-net utilizing the HWO algorithm in the ASS model reduces the computational load and shows enhanced output.
- **HWO-based U-Net model:** Hybrid optimized U-Net model with skip connection integrates the spectrogram features and the statistical features effectively for eliminating the loss of signal features. In addition, the U-Net architecture with HWO

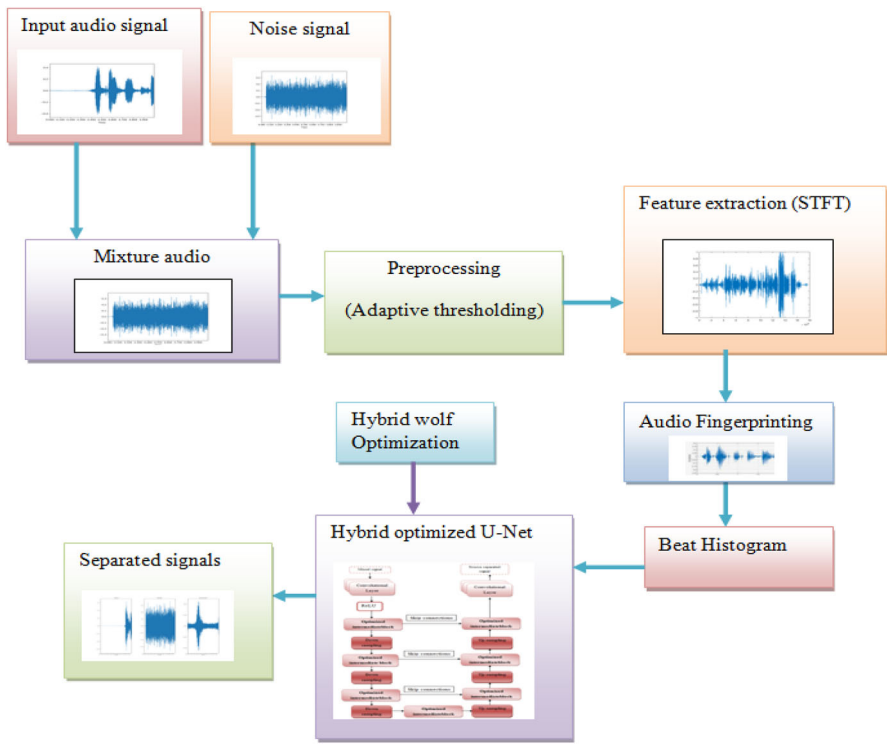


Fig. 1 Schematic representation of the proposed ASS model

trains the complex-valued spectrograms that identify both magnitude and phase, which enhances the performance of HWO-UNet. Finally, the U-Net model with HWO separates the audio source with minimum loss and reduces the problem of class balance effectively.

The manuscript's organization includes the following: Sect. 2 comprises the review of the related works, Sect. 3 elaborates on the developed method of ASS using U-Net with HWO, Sect. 5 involves the implementation results and its discussion, and finally the conclusion and future scope are included in Sect. 5.

2 Literature Survey

This part reviews existing methods of various researchers specifically, its advantages, as well as disadvantages, are highlighted in this section. Paul Magron et al. [41] presented an oMISI model that fits right for source separation of low-latency audio applications. Though MISI with DNN provided a promising ASS offline, oMISI worked online without any performance loss. The challenging issue with this method is the phase separation of the audio is not considered as well as it was evaluated only for a single-channel speech separation task.

Thomas Sgouros et al. [8] suggested a Directional fuzzy C-means framework to estimate the quantity of audio signal found in a mixture. This framework provides high performance in extracting the number of audio sources as well as separating the sources. The usage of convolutive multi-channel mixtures was a challenging approach in this paper.

Thomas Sgouros et al. [12] stated the Attentive Short-Time Discrete Cosine Transform Data were the inputs for a MultiResUNet Architecture. When compared to the conventional approaches, the model's solution was more effective and required less computing power. For more complicated networks, the method that was given did not effectively transmit information.

Bracha Laufer-Goldshtein et al. [9] presented the GLOSS algorithm which significantly increased the separation scores as well as tested under real-life recording datasets and hence it served as one of the robust methods. The disadvantage of this method was the SVM classifier, for detecting the number of speakers, which was not suitable for large datasets.

Mirco Pezzoli et al. [43] utilized a method of MNMF for ASS and also used Ray-Space-Transformed Data which provided several configurations. This method is employed with real-world recordings that provide an effective result. The major drawback of this method was it required large computational resources as well as the model sizes when increasing the number of sampling frequencies to train the SDR.

Koichi Saito and Tomohaiko Nakamura [2] used SFI structure that generated weights of the convolutional layer. This method used signal resampling while the input signals were lowered than the trained signals and also TD SFI convolutional layer used time domain sampling that increase weight generation. The disadvantage is that the computation time is higher throughout the method.

Tomohiko Nakamura et al. [44] used a model called MRDLA that did both samplings to have original time resolution. It had an anti-aliasing property and a perfect reconstruction property which improved audio separation. If the estimation in any musical instrument fails, it would affect the other instrument estimation too and this leads to performance degradation in the method.

Markus Schwabe and Micheal Heizmann [38] employed a model with a joint separation approach using U-net architecture which predicts the best separation along with high robustness. This method also provided a flexible line-up separation. The challenges observed in this method were the results found with real predictions did not provide high robustness as well as the large label error influenced the performance of the music source separation.

2.1 Challenges

The limitations of the conventional techniques are discussed below.

- DNN method typically takes the phase of each separated spectrogram to be the phase of the observed complex spectrogram; nevertheless, this can lead to an inconsistent spectrogram, meaning that using redundant time–frequency transform does not ensure the existence of a corresponding time domain signal. Finally, suboptimal results are obtained from disregarding this stage of the separation procedure.

- The disadvantage of the polynomial matrix method is that it fails to add coupling between parameter subsets. For example, the representation of the linear combinations of eigenvoices [53] or eigenbeam signals [42] as sources of characteristic spectral patterns.
- The spatial covariance method did not use the source latent features obtained by the unsupervised training for down-streaming tasks and these features can be the additional inputs to the source position provided in prior results in poor accuracy [14].
- U-net works with ADAM optimizer which can also lead to overfitting when the learning rate varies. One of the significant flaws is that it cannot separate mixed sources in large datasets and requires longer training time [27].

To overcome the aforementioned difficulties, the proposed research makes use of the U-Net model with HWO, in which UNET is an encoder-decoder network architecture that functions as an oversampling technique to address the issue of class imbalance, and the anti-aliasing method, which increases efficiency. To successfully eliminate the loss of signal features, a hybrid optimized U-Net model with skip connection includes both statistical and spectrogram information. Further, the features are fed into the U-Net model, which has been trained with the HWO algorithm that offers a high convergence rate and lowers the time complexity of the model by combining the hunt-search traits of wolves with the predator-prey traits of hawks.

3 Audio Source Separation Using HWO Algorithm-Based U-Net Model

Various methods were developed in ASS research and with those existing methods, there were several drawbacks. In [47], the author pointed out various methods such as C-BiLSTM, PSO, Secant, and Newton Raphson methods. Although this method provided a better separation performance, it lacks oversampling techniques in class balancing. To address the drawbacks of the conventional techniques, an improved ASS method is developed. The method utilizes the U-Net-based architecture with optimized intermediate spectrogram transformation blocks having adaptive tuning behavior of Deep CNN. The signal architecture of HWO-UNet is actually encoder-decoder network architecture that works as an oversampling technique that tackles the class imbalance problem, as well as the anti-aliasing method, providing more efficiency. Figure 2 explains the proposed methodology of this research, which utilizes three different datasets such as UrbanNoise8k, Librispeech, and MUSDB18 through which the architecture trains the complex-valued spectrograms that identify both magnitude and phase, which increases the performance of the HWO-UNet method. Each dataset contains noise as well as voice signals respectively and they are given as an input to the pre-processing block. Adaptive thresholding organizes raw data to build an efficient input for further steps and uses a certain threshold value to eliminate the inconsistent signals and the outputs from the preprocessing blocks are concatenated since before they act as parallel processing. Now, the concatenated output and preprocessed voice signal are fed into the feature extraction block, where feature extraction is carried out by spectrogram feature through which the speed of training the dataset

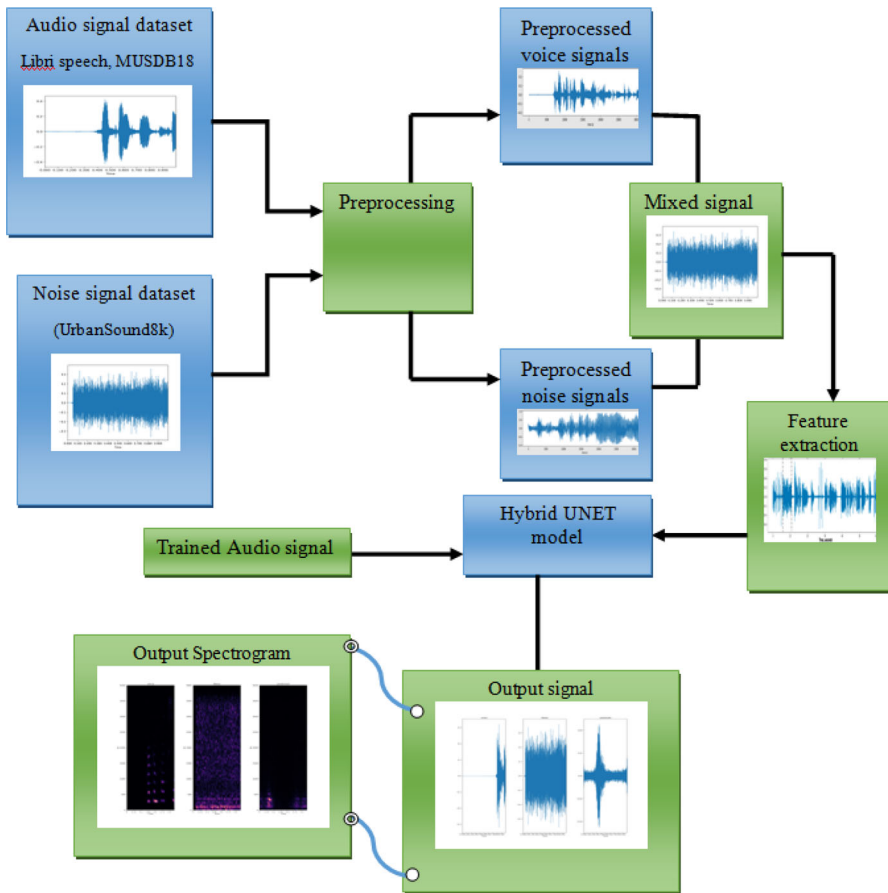


Fig. 2 Block diagram of the time-domain audio source separation with optimized U-NET

can be increased. The feature extracted mixture signal and the voice signal are given as input to the UNet model in which the sampling takes place using the improved model from which the separated voice and noise signal is obtained.

3.1 Input Signal

The research utilizes three datasets, UrbanSound8k, LibriSpeech, and MUSDB18. The UrbanSound8k [36] is the dataset of noise that contains 8732 labeled sounds of 10 classes including sounds of air conditioners, car horns, children playing, dog barking, and so on. Librispeech [15] dataset holds 1000 h of reading English speech as well and the MUSDB18 [19] is a musical dataset that includes different sounds of the musical instruments.

3.2 Preprocessing

Preprocessing is a process of organizing the raw data from the database and thus the preprocessed data is highly suited for producing an efficient result. In this stage, the audio signals are cleaned using adaptive thresholding to remove background noises and irrelevant information [34]. Adaptive thresholding is a method of amplitude-frequency relationship, which follows three steps: Firstly, the complete audio file is read with a 16 k sampling rate. Secondly, the maximum amplitude is computed and at last, the audio file is reconstructed without noise. In this research, the input to the preprocessor block is both the audio signal $X(n)$ as well as the noise signal $N(n)$ from the respective datasets, and the adaptive thresholding sets the threshold value as 1.5 s by which the signals below this threshold value are eliminated from the input signals thus, creating a clean output of audio and noise signals from the preprocessor block.

$$N^*(n) = X^*(n) = A \sin(2\pi ft) \quad (1)$$

In the Eq. (1), A represents Amplitude and f represents the frequency of the pre-processing signal. The outputs such as $N^*(n)$ and $X^*(n)$ from the preprocessing block are concatenated to form a mixed signal.

$$x[n] = [X^*(n), N^*(n)] \quad (2)$$

where, $x[n]$ is a mixed signal containing preprocessed noise as well as audio signals.

3.3 Feature Extraction

To reduce the dimension of the signal that is to be utilized for further processing from a large dataset, feature extraction is adopted in the model. In this context, the important feature of the signal is extracted with spectrogram features. Let $x[n]$ be mixed signal input to feature extraction process.

3.3.1 Spectrogram-Based Feature Extraction Using Short-Time Fourier Transform (STFT)

The spectrogram is a pictorial representation of the spectrum of signal frequencies concerning time [21]. Several techniques, including the Fourier transform, wavelet transform, and band-pass filters, can be used to create the spectrogram. In the research, the STFT is utilized to create the spectrogram and the ultimate aim of STFT is to separate a signal into shorter segments of equal length [21] in which Fourier transform is applied and their respective result is placed in the time–frequency spectrogram [5]. Processing the spectrogram feature is expensive, as it carries complex values. The magnitude is therefore frequently employed in processing and separation. The inverse STFT must be used to return the separated output to the time domain after separation [5]. However, to identify the phase of the source, many methods are carried out and all these methods require inversion of signal to the time domain to reconstruct the signal

later. In the research, the Spectrogram feature is extracted for the preprocessed voice signal as well as the mixed signal containing voice and noise, and these extracted mixed signals and the voice signal are given as a feature and label to train the U-Net model. STFT of the input signal is represented as, $Y(\tau, \omega)$

$$Y(\tau, \omega) = \int_{-\infty}^{\infty} x[n] w(n - \tau) e^{-i\omega n} dn. \quad (3)$$

where (τ, ω) are the time–frequency atoms and $w(n)$ is the Gaussian window with finite support. Now, the spectrogram function is represented as the square of the STFT function $Y(\tau, \omega)$

$$\text{Spectrogram}\{x[n]\}(\tau, \omega) = S_p = |Y(\tau, \omega)|^2 \quad (4)$$

Let us assume the feature acquired from the STFT is denoted as, S_p .

3.3.2 Spectrogram-Based Extraction of Features Utilizing MFCC

Mel-Frequency Cepstral Coefficient (MFCC) is the best algorithm used in the audio signal domain, which compresses the information to a smaller number of coefficients to proceed with the process without complexity. MFCC feature extraction comprised of framing, windowing, FFT, Mel filter bank, and DCT. The first step is the output from the preprocessing block is given as the input and here the signals are trimmed into equal sections called frames and also the samples in the frames (N) are separated with M number of samples with the condition $M < N$. The windowing process is the next step in which frames are structured by minimizing the disruptions at the initial and final of the frames. There are various types of windows such as Flat windows, Rectangular windows, Hamming windows[12], and so on in which Hamming window is usually utilized for audio signals and is represented as

$$H[n] = 0.54 - 0.4 \cos\left(\frac{2\pi n}{N-1}\right) \quad (5)$$

where $H[n]$ is the Hamming coefficient, N , is the sample count in the frame. The output of the windowing process is given as

$$y[n] = x[n] \times H[n]. \quad (6)$$

where, $x[n]$ is the input signal from the preprocessing stage. Subsequently, it proceeds to the Fast Fourier Transform stage, which serves as the Discrete Fourier Transform (DFT) fast procedure by converting domains from time to frequency. Further, the representation of DFT as follows

$$D_f = \sum_{n=0}^{N-1} y[n] e^{-\frac{j2\pi kn}{N-1}} \quad (7)$$

where $k = 0, 1, 2, \dots, N - 1$, D_f is the frequency pattern generated using Fourier transform and is called Spectrum. Mel filter bank, in which the spectrums are mapped in the triangular filter bank. Usually, Mel frequency is low frequency since it works as the human ear audio scale and its logarithm generates a maximum value that is above 1000 Hz [1]. The Mel frequency is formulated as follows,

$$Mel_f = 2595 \log_{10} \left(\frac{f_h}{700} + 1 \right) \quad (8)$$

where, f_h represents the frequency in hertz and Mel_f denotes the Mel frequency. It becomes easy to estimate the energies at the spot with the filter bank as well as the proper spacing with Mel scaling. This energy estimation followed by logarithm is called Mel spectrum and it can be used for calculating the first 13 coefficients using DCT. The process of DCT is carried out to reconvert the logarithmic function into a time domain [12]. Usually, DCT works with a smaller amount of coefficients and so requires less memory storage to represent the Mel spectrum [1]. The output obtained from DCT is Mel Frequency Cepstral Coefficient.

$$C_c = \sum_{k=1}^N (\log D_f) \cos[n(k - 0.5)\pi / N] \quad (9)$$

where C_c denotes the Mel Frequency Cepstral Coefficient and n is the number of coefficients. Let us assume the feature acquired from the MFCC is represented as, C_c .

3.3.3 Statistical Features

The bands represent the signal energy distribution in the time–frequency domain and statistical features held up to 4th order include the mean, variance, skewness, and Kurtosis [7]. Along with this, the entropy is also evaluated to establish the feature vector that ensures the accuracy of audio separation.

(a) Mean: The mean the signal is the arithmetic average of the signals from 1 to N , around which central clustering of signals occurs.

$$Mean \quad \mu = \frac{1}{N} \sum_{j=1}^N x_j, \quad (10)$$

where N denotes the number of signals.

(b) Variance

The variance V_v is the mean signal square and is derived as follows,

$$V_v = \frac{1}{N} \sum_{j=1}^N (x_j - \mu)^2 \quad (11)$$

where, N is the number of signals, and μ is the mean signal.

(c) Skewness: The evaluation of symmetry around the signalspectrum[31] is known as skewness and is computed as follows,

$$\text{Skewness } Sk = \frac{1}{N} \sum_{j=1}^N \left[\frac{x_j - \mu}{\sigma} \right]^2 \quad (12)$$

where, N denotes the number of signals, σ denotes the standard deviation, and μ is the mean.

(d) Kurtosis: The fourth-order statistical evaluation that measures the peak distribution of the signal [7] is known as kurtosis.

$$\text{Kurtosis } KT = \frac{1}{N} \sum_{j=1}^N \left[\frac{x_j - \mu}{\sigma} \right]^4 \quad (13)$$

(e) Entropy: The measure of the quantity of information found and the degree of unstable signals. It also serves as a first approximation for the system disturbance in audio signals [30].

$$\text{Entropy } E = -\text{sum}(V * \log(V)) \quad (14)$$

where V denotes the probability vector. Let S_f be the statistical features.

3.3.4 Audio Fingerprinting

Audio fingerprinting refers to a technique that uses a highly discriminative feature termed the "fingerprint" [31] to identify a small portion of audio from a large collection and often condenses the audio recordings. The messages are automatically derived using this method from the most pertinent audio elements. The audio fingerprinting technologies that depend on the application used should be able to detect the distortions present in the query audio material. The minimum requirements for audio fingerprinting are the robustness of the various methods of lossy audio compression and the speed at which big databases may be processed [4]. These fingerprinting techniques are typically used in media monitoring, which involves routinely analyzing the audio streams of a large number of broadcast channels [31]. The descriptor, which increases feature robustness and decreases distortions like noise and speed change, is used to deal with fingerprinting. Let us assume the features acquired from the audio fingerprinting are denoted as, A_f .

3.3.5 Beat Histogram

The signal envelope is produced for the sample signal, and the frequency and amplitude are estimated using a beat histogram, called beat spectrum [26]. The signal envelope's autocorrelation is then calculated. Then, a beat histogram is extracted in the peaks of

the signal which is considered as one of the major features. Let us assume the features acquired from Audio fingerprinting are denoted as, B_h .

The feature vector extracted is represented as $F_v = [S_p || C_c || S_f || A_f || B_h]$, and the dimension of the feature vector is denoted as, $[N \times 128 \times 128 \times 1]$, where N is the total number of input signals.

3.4 ASS with HWO-Optimized U-Net Model

The research utilized U-Net architecture, which is of encoder-decoder architecture, in which the feature map time resolution is halved by the encoder via downsampling the feature maps using a series of down-sampling (DS) blocks, and doubled the resolution time by the decoder through an upsampling the feature maps through the series of up-sampling (US) blocks. Utilized U-Net architecture follows a standard design and is comprised of 24 layers including the input, convolutional, and both downstream and upstream sections. The unique feature of U-Net is that the dimensions provided as input remain the same at the output layer. Along the downstream path, a factor called 2-decimation is followed in the convolutional layer to retain the high resolution. Similarly, in the upstream path, to enhance up-sampling, a bilinear interpolation in the 1-D convolutional layer is carried out. The skip connections are provided between optimization blocks of downstream and upstream paths. The large number of filters is accompanied by the flow of information in the path of U-Net, which makes U-Net architecture more advantageous. Every convolutional layer included the Leaky ReLU activation function [29]. In the research, U-Net acts as a detection model with standard U-Net architecture and includes the optimized intermediate blocks, which are comprised of Deep CNN and hybrid optimization. Deep CNNs are a type of traditional feed-forward neural network that employs BP algorithms to modify parameters to minimize cost function value [45], which is the MSE function for this research. The input dimension of this method is $[N \times 128 \times 128 \times 1]$ since it accepts 4 dimensional characteristics.

(a) Convolutional layer: Convolutional layer acts as the core layer of the deep CNN, which requires input, kernels as well as feature maps. The convolution layer transforms the input into low-level and high-level components of the extracted pattern.

$$i_j = \sum_{j=0}^K W_j^{b-1} * F_v + b_j^b \quad (15)$$

where, and W_j^{b-1} is the weight applied in the signal with $b - 1$ number of kernels. The bias of j^{th} feature is indicated by b_j , and F_v is a feature vector that acts as an input.

(b) Activation function layer: The most important layer is the ReLU, which contributes to the non-linearity property. The resulting convolutional layer maps are transmitted via the non-probability layer (ReLU layer), which does not have any weight

or bias. For the activation function, the mathematical procedure is as follows,

$$\text{ReLU}(i) = \begin{cases} 0, & \text{if } i < 0 \\ i, & \text{if } i \geq 0 \end{cases} \quad (16)$$

All layer neurons are fully associated with the activation mechanisms of the layer below.

(c) Max Pooling Layer: This layer contains the down-sampled feature maps produced by convolutional layers to reduce the spatial dimension and the amount of information.

(d) Skip connections: The skip connections are the connection between the encoder and decoder networks, which helps U-Net architecture not to skip even fine-grained information regarding the signals from the encoder section to the decoder section. These connections are said to be long connections whereas the connections found in ResNet are short. The decoder layer directly receives the feature map U_{en}^i while the intra-decoder skip connections transmit a larger scale of the decoder layer U_{de}^i [11] and it is represented as,

$$U_{de}^i = \begin{cases} U_{en}^i, & i = E \\ R \left(\left[\underbrace{C(P(U_{en}^m))_{m=1}^{i-1}, C(U_{en}^i)}_{\text{scales: } 1^{th} - i^{th}}, \underbrace{C(Q(U_{de}^m))_{m=i+1}^E}_{\text{scales: } (i+1)^{th} - E^{th}} \right] \right), & i = 1, \dots, E-1 \end{cases} \quad (17)$$

where $C(\cdot)$ is the convolution function, $R(\cdot)$ represents the feature aggregation mechanism along with batch normalization and ReLU activation function, $P(\cdot)$ and $Q(\cdot)$ represents the down sampling as well as the upsampling, respectively. The architecture of the U-Net model is depicted in Fig. 3.

Further, the hyperparameters of the developed UNet are optimized utilizing the Hybrid Wolf Optimization, which is explained in the below section.

The layer details of the developed UNet are depicted in the Table 1 as follows.

3.5 Proposed Hybrid Wolf Optimization

In the existing models such as U-net works with Adam optimizer which can also lead to overfitting when the learning rate varies. However, the proposed research, in which suggested study, in which the overfitting problem is mitigated by training the U-Net using the high accuracy value found by employing the hybrid wolf optimization algorithm's best solution after several iterations. The Hybrid Wolf Optimizer (HWO) is designed by the hunt-search characteristics of the wolves [18] and predator–prey characteristics of the hawks [33] to train the audio source separator to acquire better accuracy. The HWO approach offers several benefits over the usual optimization technique. Firstly, it addresses the issue of premature convergence during the exploration

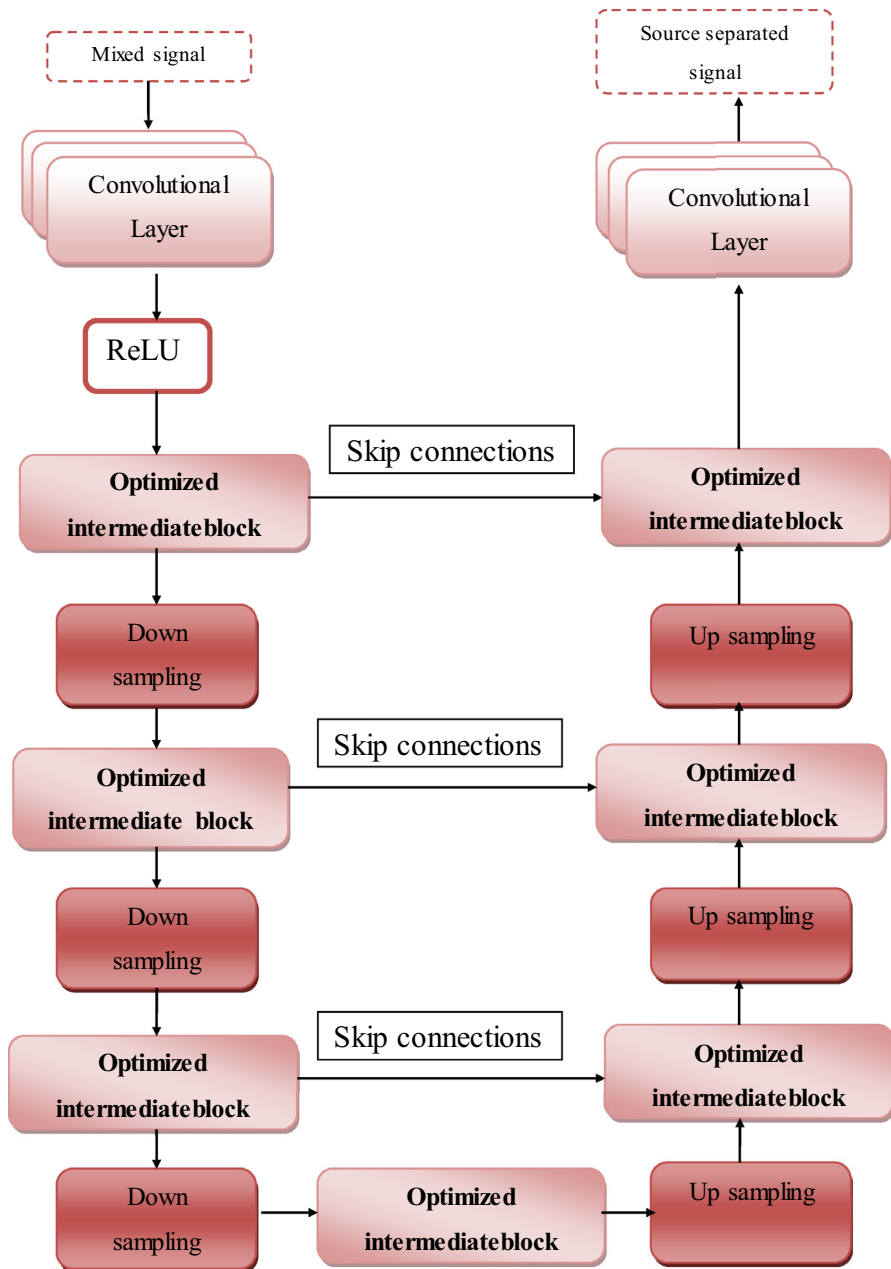


Figure 3: Architecture of the U-Net Model

Fig. 3 Architecture of the U-Net model

Table 1 Layer details of the U-Net model

Layer	Details	Output size
input	Input size: $128 \times 128 \times 1$	$128 \times 128 \times 1$
conv_2d	$128 \times 128 \times 1$; dilation rate = 1,1; filters = 16; kernel size = 3,3; Leaky ReLU	$128 \times 128 \times 16$
conv_2d_1	$128 \times 128 \times 16$; dilation rate = 1,1; filters = 16; kernel size = 3,3; Leaky ReLU	$128 \times 128 \times 16$
max pool_2d	$128 \times 128 \times 16$; pool size = 2,2; strides = 2,2	$64 \times 64 \times 16$
conv_2d_2	$64 \times 64 \times 16$; dilation rate = 1,1; filters = 32; kernel size = 3,3; Leaky ReLU	$64 \times 64 \times 32$
conv_2d_3	$64 \times 64 \times 32$; dilation rate = 1,1; filters = 32; kernel size = 3,3; Leaky ReLU	$64 \times 64 \times 32$
max pool_2d_1	$64 \times 64 \times 32$; pool size = 2,2; strides = 2,2	$32 \times 32 \times 32$
conv_2d_4	$32 \times 32 \times 32$; dilation rate = 1,1; filters = 64; kernel size = 3,3; Leaky ReLU	$32 \times 32 \times 64$
conv_2d_5	$32 \times 32 \times 64$; dilation rate = 1,1; filters = 64; kernel size = 3,3; Leaky ReLU	$32 \times 32 \times 64$
max pool_2d_2	$32 \times 32 \times 64$; pool size = 2,2; strides = 2,2	$16 \times 16 \times 64$
conv_2d_6	$16 \times 16 \times 64$; dilation rate = 1,1; filters = 128; kernel size = 3,3; Leaky ReLU	$16 \times 16 \times 128$
conv_2d_7	$16 \times 16 \times 128$; dilation rate = 1,1; filters = 128; kernel size = 3,3; Leaky ReLU	$16 \times 16 \times 128$
dropout	$16 \times 16 \times 128$, rate = 0.5	$16 \times 16 \times 128$
max pool_2d_3	$16 \times 16 \times 128$; pool size = 2,2; strides = 2,2	$8 \times 8 \times 128$
conv_2d_8	$8 \times 8 \times 128$; dilation rate = 1,1; filters = 256; kernel size = 3,3; Leaky ReLU	$8 \times 8 \times 256$
conv_2d_9	$8 \times 8 \times 256$; dilation rate = 1,1; filters = 256; kernel size = 3,3; Leaky ReLU	$8 \times 8 \times 256$
dropout_1	$8 \times 8 \times 256$; rate = 0.5	$8 \times 8 \times 256$
up_sampling_2d	$8 \times 8 \times 256$; interpolation = nearest size = 2,2	$16 \times 16 \times 256$
conv_2d_10	$16 \times 16 \times 256$; dilation rate = 1,1; filters = 128; kernel size = 2,2; Leaky ReLU; concatenate output: $16 \times 16 \times 256$	$16 \times 16 \times 128$
conv_2d_11	$16 \times 16 \times 256$; dilation rate = 1,1; filters = 128; kernel size = 3,3; Leaky ReLU	$16 \times 16 \times 128$
conv_2d_12	$16 \times 16 \times 128$; dilation rate = 1,1; filters = 128; kernel size = 3,3; Leaky ReLU	$16 \times 16 \times 128$
up_sampling_2d_1	$16 \times 16 \times 128$; interpolation = nearest size = 2,2	$32 \times 32 \times 128$

Table 1 (continued)

Layer	Details	Output size
conv_2d_13	$32 \times 32 \times 128$; dilation rate = 1,1; filters = 64; kernel size = 2,2; Leaky ReLU, concatenate_1 output: $32 \times 32 \times 128$	$32 \times 32 \times 64$
conv_2d_14	$32 \times 32 \times 128$; dilation rate = 1,1; filters = 64; kernel size = 3,3; Leaky ReLU, concatenate	$32 \times 32 \times 64$
conv_2d_15	$32 \times 32 \times 64$; dilation rate = 1,1; filters = 64; kernel size = 3,3; Leaky ReLU	$32 \times 32 \times 64$
up_sampling_2d_2	$32 \times 32 \times 64$; interpolation = nearest size = 2,2	$64 \times 64 \times 64$
conv_2d_16	$64 \times 64 \times 64$; dilation rate = 1,1; filters = 32; kernel size = 2,2; Leaky ReLU; concatenate_2 output: $64 \times 64 \times 64$	$64 \times 64 \times 32$
conv_2d_17	$64 \times 64 \times 64$; dilation rate = 1,1; filters = 32; kernel size = 3,3; Leaky ReLU,	$64 \times 64 \times 32$
conv_2d_18	$64 \times 64 \times 32$; dilation rate = 1,1; filters = 32; kernel size = 3,3; Leaky ReLU	$64 \times 64 \times 32$
up-sampling-2d-3	$64 \times 64 \times 32$; interpolation = nearest size = 2,2	$128 \times 128 \times 32$
conv_2d_19	$128 \times 128 \times 32$; dilation rate = 1,1; filters = 16; kernel size = 2,2; Leaky ReLU, concatenate_3 output: $128 \times 128 \times 32$	$128 \times 128 \times 16$
conv_2d_20	$128 \times 128 \times 32$; dilation rate = 1,1; filters = 16; kernel size = 3,3; Leaky ReLU	$128 \times 128 \times 16$
conv_2d_21	$128 \times 128 \times 16$; dilation rate = 1,1; filters = 16; kernel size = 3,3; Leaky ReLU	$128 \times 128 \times 16$
conv_2d_22	$128 \times 128 \times 16$; dilation rate = 1,1; filters = 2; kernel size = 3,3; Leaky ReLU	$128 \times 128 \times 2$
conv_2d_23	$128 \times 128 \times 2$; dilation rate = 1,1; filters = 1; kernel size = 1,1; activation = tanh	$128 \times 128 \times 1$

phase of the wolf's hunting characteristics [17], resulting in increased accuracy. Additionally, it recognizes the prominent features in the signal without compromising the accuracy range. Furthermore, HWO computational time is far less than the standard optimizations even in seconds, and also considers the energy factor that balances the global exploration as well as the local exploitation [33].

3.5.1 Inspiration

The Hunt-search characteristics [18] are inspired by *Canis Lupus* of the family *Canidae*, which lives in the pack having the best social hierarchy as well as the intelligent hunting method. The social hierarchy contains the leader named alpha and the subordinates beta, delta, and omega. Group hunting with efficient steps like searching, encircling, and attacking the prey is the intelligent hunting method of *Canis Lupus*.

Usually, they diverge from the group to search for prey and converge to hunt the prey is an advantageous characteristic of the pack. The next characteristic of the research is predator–prey characteristics [33], which are inspired by the *Parabuteo unicinctus*, generally known as Harris Hawk. The hunting behavior in the exploration and the exploitation phase along with a cooperative behavior to attack the escaping prey with various strategies serve as the unique characteristics. The major part of the hunting behavior is updating the hunting strategies of the group based on the varying energy levels and the escaping patterns of the prey. Further, the characteristics of both traditional optimizations yield better results, the hybrid method is more likely to exclude the local optimal solutions, increasing accuracy in the process. There is a discussion of the stages involved in the HWO in Fig. 3.

(a) Initialization of solution: The first stage of optimization is initializing the solution, which allows solutions to be declared. The search solutions are initialized as, J , which is nothing but the tunable parameters of the U-net model.

(b) Objective function: The objective function of the solution is represented as, $O(S)$, which relates with the loss function of the solution as,

$$O(S) = MSE(\text{Solution } S) \quad (18)$$

The solution corresponding to the minimal MSE is declared as the best solution for tuning the U-net model.

(c) Solution Update: Every iteration ends with an update and renovates the solution to identify the global best solution, for which the following update guidelines and procedures are followed in the optimization.

Case 1: $S > 0.9$ —**Exploration phase:** When the objective solution is greater than 0.9 then move towards the exploration phase, where the individual hunt phase is initiated. In the exploration phase, the process of perching and detecting the solution takes place.

Individual Hunt Phase: In this phase, the search is enabled in different directions within a boundary to locate the best solution and the hunt is carried out based on the convergence factor of the solutions. The best position on the perch is updated continuously.

The phase can be classified based on the convergence factor called q .

Case (a): Convergence factor $q \geq 0.5$. This case enables the random search throughout the boundary to predict the solution. The predator perches randomly in some locations or on the location near the other predator and waits for a while to observe the best solution in each step.

$$J^{tH} = 1/2 \{ J_{rand}^t - r_1(J_{rand}^t - 2r_2.J^t) + J_{seq} - Q.M \} \quad (19)$$

$$J^{tH} = 1/2 \{ J_{rand}^t(1 - r_1) - 2r_2.J^t + J_{seq} - Q.M \} \quad (20)$$

where, J_{rand}^t is the random solution vector, r_1 and r_2 are random vectors, J^t is the current solution, and J_{seq} is the sequence solution based on the randomly formed solution Q , M are the coefficient vectors.

Case (b): Convergence factor $q < 0.5$

The bounced search happens, where the search moves deep to predict the solution with minimum fitness value and the random scaling coefficient provides more diversification trends and explores different regions.

$$J^{tH} = (J_p^t - J_{seq}) - r_3(R + r_4(U_p - L_p)) \quad (21)$$

where, J_p^t is the position of the prey, U_p, L_p are the upper and lower population of the boundary through which the position of the solution is generated for the hunting process, R , is the random scaling coefficient.

Case2: Exploitation Phase- $\$ \leq 0.9$

When the objective solution falls below 0.9, moves to the exploitation phase having other three sub-phases with each condition namely the bouncing phase, the fragile phase, and the trait phase.

Bouncing Phase: $e \geq 0.5$ & $\eta \geq 0.5$

The Bouncing phase is where the solution is found with the maximum capability factor but fails to implement the escaping mechanisms or there is no available escaping mechanism and the predator finds the best search strategies for non-destructive foraging. Though the predator encircles the prey softly, the prey gets exhausted.

$$J^{tH} = \gamma_1(J_p^t - J^t)(1 - \eta) + \gamma_1(J^{t-1} - J^t) \quad (22)$$

where, e is the convergence probability, η is the Solution capability factor, and γ is the fusion probability with $\gamma_1 = 0.5$. Also J^{t-1} is the solution of the previous instance.

Fragile Phase: $e \geq 0.5$ & $\eta < 0.5$

The solutions in this phase found with low energy but still establish some escaping patterns to escape from the predator. These escaping patterns are in zigzag movement which made the predator use LF-based activities to enhance the hunting tactics which are based on the diving movements of the predator.

$$J^{tH} = J_p^t(1 - \eta) + J^t(\eta - \gamma_2 + \gamma_3) + \gamma_2 \cdot J_g^t - \gamma_3 J_{pe}^t \quad (23)$$

where J_p^t is the prey solution, J_g^t is the global best solution, and J_{pe}^t is the personal best solution.

Trait Phase: $\eta \geq 0.5$ & $e < 0.5$: In the Trait phase, the predators update the location in escaping patterns of the solutions and if the escaping patterns are lost or found abrupt the predators perform various movements to update the current location and decrease the distance between the solution and the predator.

$$J^{tH} = 1/4 \left\{ J_p^t - \eta(J_p^t - J^t) + J_1^{t-1} - Q_1 \cdot \delta_1 + J_2^{t-1} - Q_2 \cdot \delta_2 + J_3^{t-1} - Q_3 \cdot \delta_3 \right\} \quad (24)$$

J_1^{t-1}, J_2^{t-1} and J_3^{t-1} are the first three best solutions. Among all phases, the final suitable phase is utilized to enhance the efficacy of the research model. Figure 4 depicts the flowchart for the HWO algorithm.

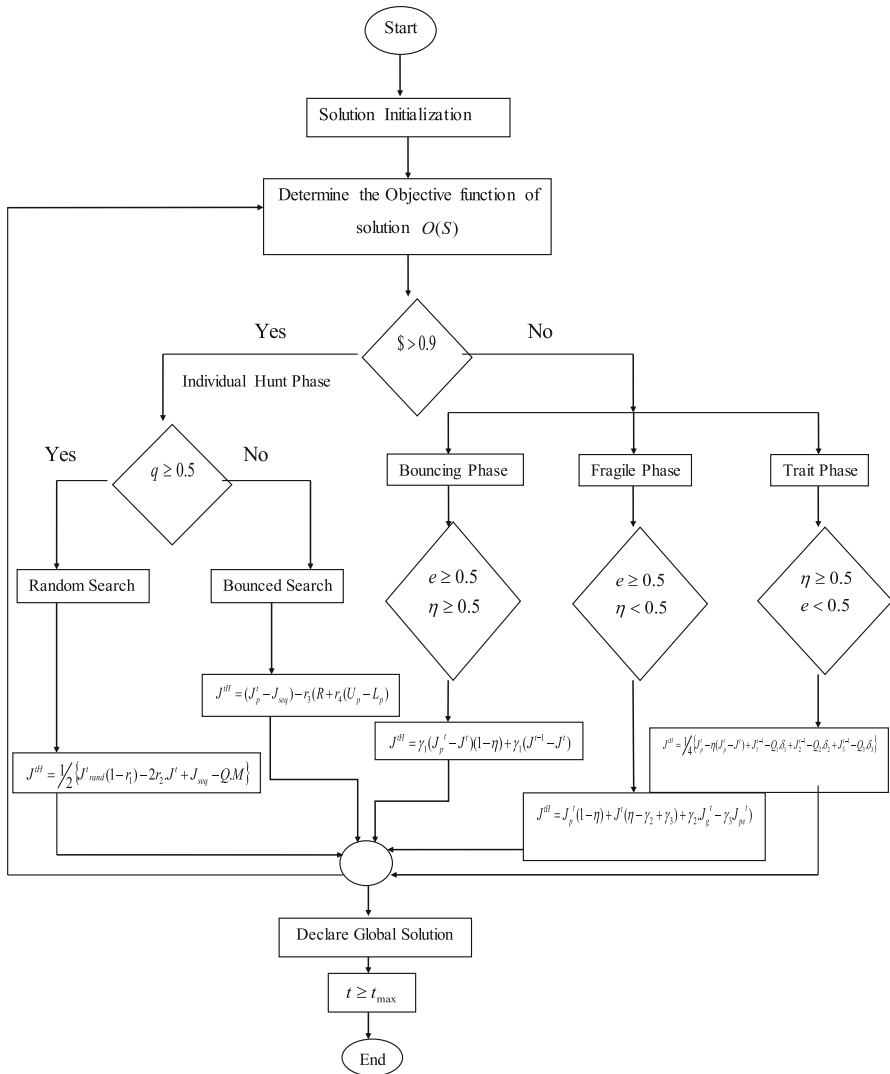


Fig. 4 Flowchart of HWO algorithm

4 Results and Discussion

In the section below, the efficiency of the HWO-UNet model for ASS is measured with respective existing methods in different epochs of extracted features.

Table 2 Parameters of the developed framework

Parameters	Values
Sample rate	8000
Batch size	28
Learning rate	0.01
Epochs	50
Activation function	tanh
Population size	100
Loss function	MSE
Default optimizer	Adam

4.1 Experimental Details

This experiment is performed in a system having 8 GB RAM and a Windows 10 Operating System, which utilized PYTHON as a tool to implement it. The hyperparameters of the developed framework for ASS are displayed in Table 2.

4.2 Dataset Representations

(a) UrbanSound8k Dataset

The UrbanSound8k [36] is the dataset of noise that contains 8732 labeled sounds of 10 classes and These classes are derived from the sounds of real-world entities such as air conditioners, car horns, playing children, barking dogs, drilling, engine idling, gunshot, jackhammer, siren, and street music. This dataset acts as a training dataset as well this is the only dataset of noise used throughout the research.

(b) LibriSpeechdataset

The LibriSpeech [15] dataset holds 1000 h of English speech, especially book reading and it is sampled at 16 kHz. Though LibriSpeech contains various data modules, dev clean and train clean are used in this research. The dataset comprises 3 classes to train the model, 2 classes to test the model, and 2 classes to validate the model. All the sounds included in both datasets are collaged completely producing a noisy signal.

(c) MUSDB18 Dataset

MUSDB18 [19] is a musical dataset that contains the sounds of various musical instruments. This dataset carries both training as well as test folders. Train and test folders comprise 100 and 50 tracks respectively and these tracks are derived from various sources such as the DSD100 dataset, MedleyDB, Native instruments, and Canadian rock band. The total duration utilized by this dataset is almost 10 h and the signals are stereophonic that are encoded at 44.1 kHz.

4.3 Experimental Analysis

The experimental analysis section contains the outcomes of each step throughout ASS with HWO-UNet and that is depicted in Fig. 5.

4.4 Evaluation Metrics

The metrics aided in analyzing the performance of the HWO-UNet model for ASS are provided below,

(a) Mean Square Error

The average square of the discrepancy between the observed value and the model's predicted value is known as the Mean Square Error (MSE).

$$MSE = \frac{\sum_{q=1}^d (ov_q - pv_q)^2}{d} \quad (25)$$

where ov_q the observed value at is the q^{th} data point, pv_q is the predicted value at the same data point and d is the number of data points.

(b) Root Mean Square Error

RMSE represents the calculation of the square root of the mean square error (MSE), the regression error. The RMSE values' output can have values in the same units as the input variable and range from zero to positive infinity.

$$RMSE = \sqrt{\frac{\sum_{q=1}^d (ov_q - pv_q)^2}{d}} \quad (26)$$

where ov_q the observed value at is the q^{th} data point, pv_q is the predicted value at the same data point and d represents the number of data points.

(c) Signal-to-Noise Ratio

The signal-power-to-noise power ratio is known as the signal-to-noise ratio (SNR), which is evaluated in decibels and represented as

$$SNR = \frac{P_s}{P_n} dB \quad (27)$$

where, P_s and P_n are signal power and noise power respectively.

4.5 Performance Evaluation

In this part, the performance of ASS with the HWO-UNet model is completely analyzed with the metrics such as MSE, RMSE, and SNR and this analysis varies based on the three datasets utilized in this research. The performance analysis throughout

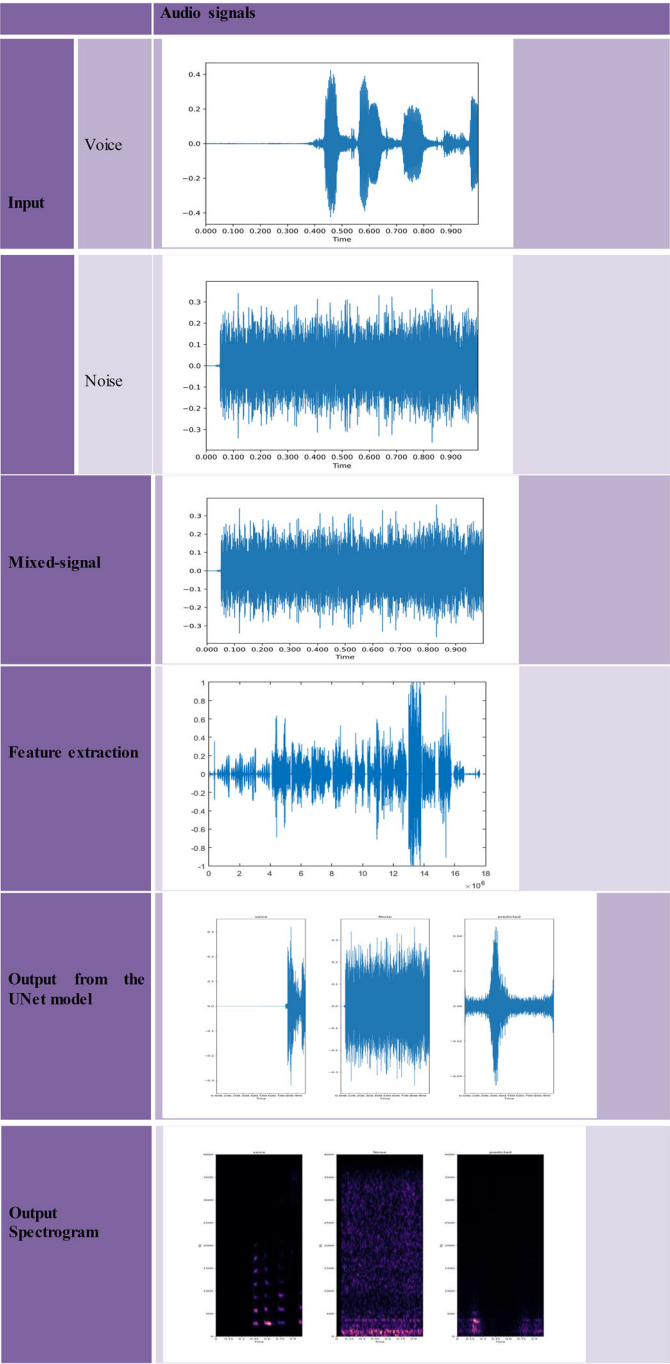


Fig. 5 Experimental results in ASS with HWO-Unet

the research is based on Training Percentage (TP) which remains constant at 90% throughout the performance analysis to enable a better view of comparison among datasets and the varying epochs.

4.5.1 Performance Evaluation with UrbanSound8k

In Fig. 6, the performance analysis of HWO-UNet for UrbanSound8K is compared with MSE, RMSE, and SNR. At 90% of training, the corresponding values of parameters MSE, RMSE, and SNR vary concerning epochs at 100, 200, 300, 400, and 500 are measured. The MSE in Fig. 6a falls to 6.668, 6.125, 5.659, 5.652 and 1.622 at respective epochs. The RMSE is 25.822 at epochs = 100, 24.75 at epochs = 200, 23.788 at epochs = 300, 23.775 at epochs = 400 and 13.142 at epochs = 500 in Fig. 6b. The SNR ratio with epochs 100, 200, 300, 400, and 500 are 7.38 dB, 9.48 dB, 10.96 dB, 11.17 dB, and 17.27 dB respectively are depicted in Fig. 6c and the analysis with UrbanSound8K shows that the maximum efficiency attains at TP = 90% and epoch = 500.

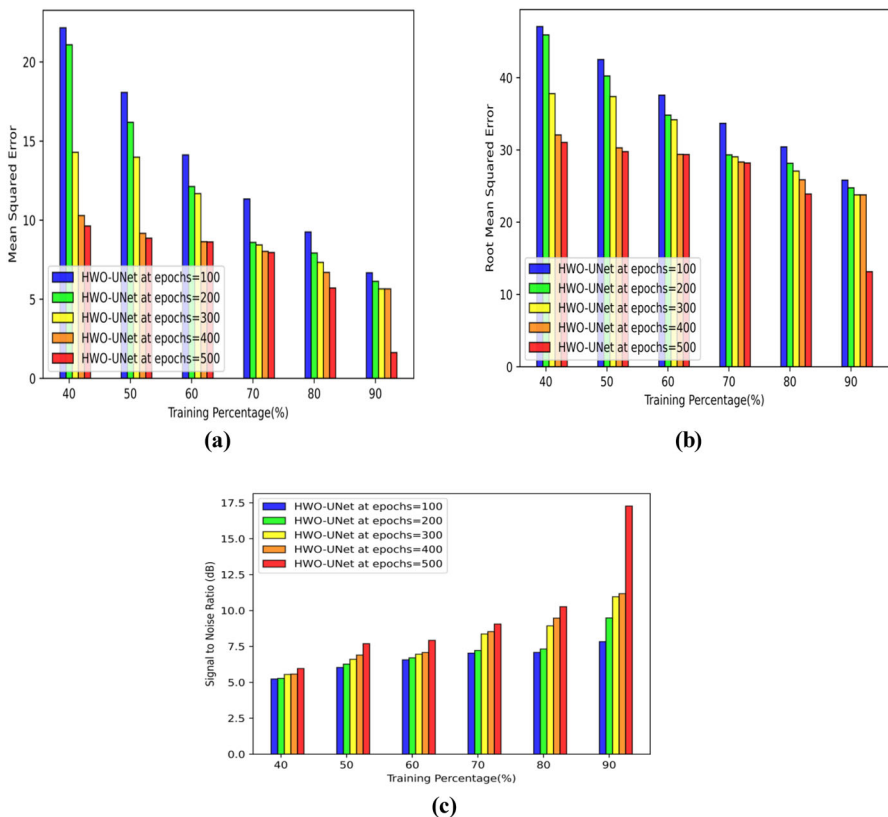


Fig. 6 Performance analysis for UrbanSound8K, **a** MSE, **b** RMSE, **c** SNR

4.5.2 Performance Evaluation with Librispeech

Performance analysis of HWO-UNet with Dataset 2 is presented in Fig. 7 which particularly specifies the performance of the MSE, RMSE, and SNR. The further comparison is based on the training percentage (TP) which remains constant at 90% and the parameters vary concerning the epochs 100, 200, 300, 400, and 500. In Fig. 7a, the MSE attains 12.15, 8.922, 8.80, 7.84 and 1.646 at respective epochs. While RMSE depicted in Fig. 7b reduces 34.85, 29.87, 29.66, 28.011, and 12.24 respectively at varying epochs 100, 200, 300, 400 and 500, the SNR in Fig. 7c enhances by 8.902 dB at epochs = 100, by 8.962 dB at epochs = 200, by 10.33 dB at epochs = 300, by 12.03 dB at epochs = 400 and reaches maximum by 16.97 dB at epochs = 500.

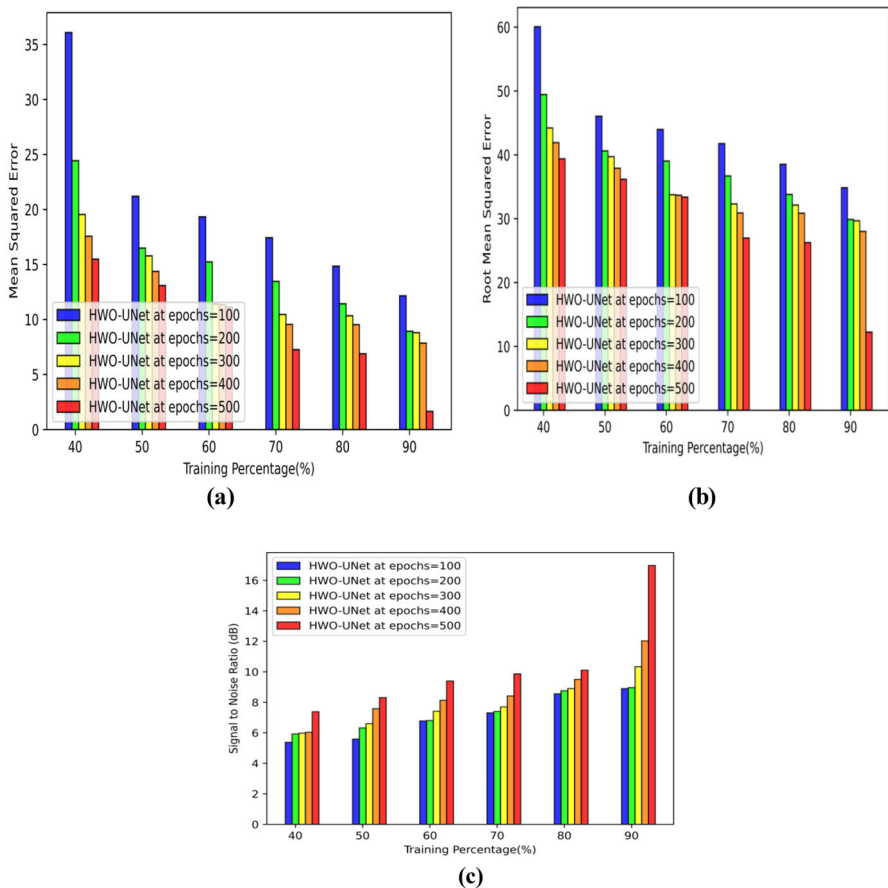


Fig. 7 Performance evaluation for Librispeech, **a** MSE, **b** RMSE, **c** SNR

4.5.3 Performance of U-Net Based on the HWO with MUSDB18

The MUSDB dataset is analyzed with parameters such as MSE, RMSE, and SNR to evaluate the performance of HWO-UNet and the complete evaluation is depicted in Fig. 8. The further comparison parameters vary concerning the epochs. In Fig. 8a, the MSE attains 7.74 at epochs = 100, 5.44 at epochs = 200, 3.52 at epochs = 300, 1.642 at epochs = 400 and 0.851 at epochs = 500. While RMSE depicted in Fig. 8b reduces 5.811, 4.505, 2.203, 1.334 at epochs = 400 and 1.186 at epochs = 500, the SNR in Fig. 8c enhances by 5.05 dB at epochs = 100, by 8.522 dB at epochs = 200, by 9.7 dB at epochs = 300, by 13.44 dB at epochs = 400, by 15.83 dB at epochs = 500. As a result, the HWO-UNet method's performance yields effective outcomes in all used datasets.

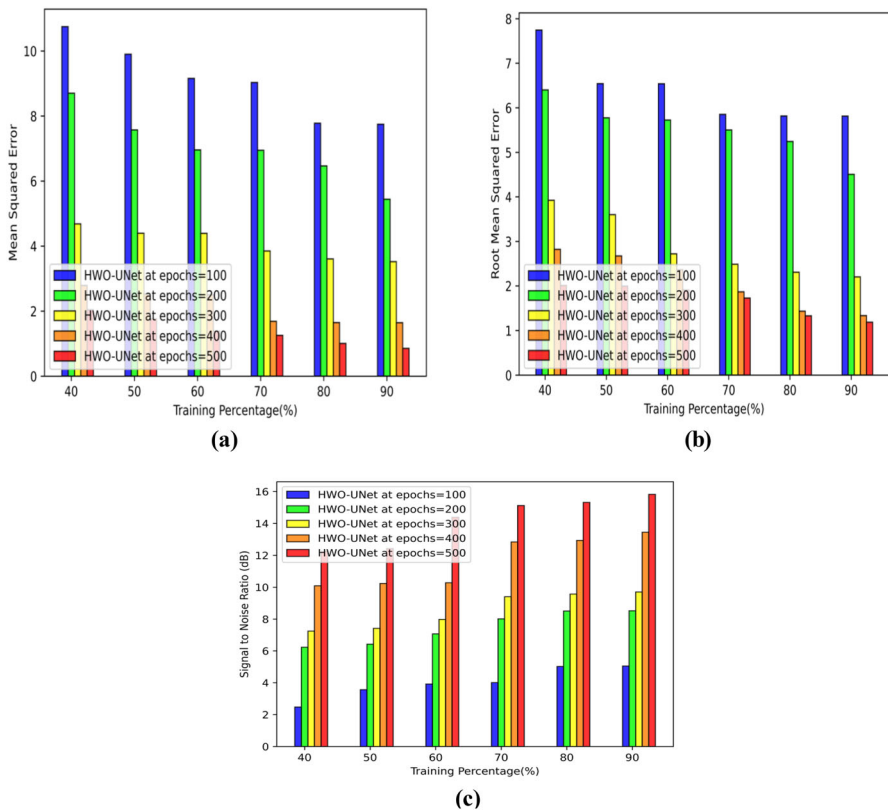


Fig. 8 Performance evaluation for MUSDB, **a** MSE, **b** RMSE, **c** SNR

4.6 Comparative Evaluation

The Comparative evaluation of the HWO-UNet method is an evaluation of HWO-UNet parameters with some traditional methods such as GRU [40], LSTM [20], UNet [27], CDE-HTCN [6], BSRNN [16], HHO-UNet [33], GWO-UNet [18].

4.6.1 Comparative Evaluation of U-Net Based on the HWO with UrbanSound8K

Figure 9 depicts the comparative analysis conducted on UrbanSound8K and the outcome is related to MSE, RMSE as well as the SNR. The MSE comparison of the HWO-UNet approach with prior methods is as follows: the HWO-UNet method attains the MSE of 1.402, which shows error differences of 0.26, 0.21, 0.18, 3.09, 0.42, 0.16, and 0.11 over the conventional techniques such as GRU, LSTM, UNet, CDE-HTCN, BSRNN, HHO-UNet, and GWO-UNet respectively. Meanwhile, the RMSE of HWO-UNet is 11.841, which shows an error difference of 1.05 to GRU. However, for LSTM, UNet, CDE-HTCN, BSRNN, HHO-UNet, and GWO-UNet, the error differences are 0.87, 0.73, 10.95, 1.33, 0.65, and 0.47 respectively. Further, the HWO-UNet method attains the SNR value of 16.51 dB, which shows the improvement percentage for the techniques GRU, LSTM, UNet, CDE-HTCN, and BSRNN by 64.76%, 54.33%,

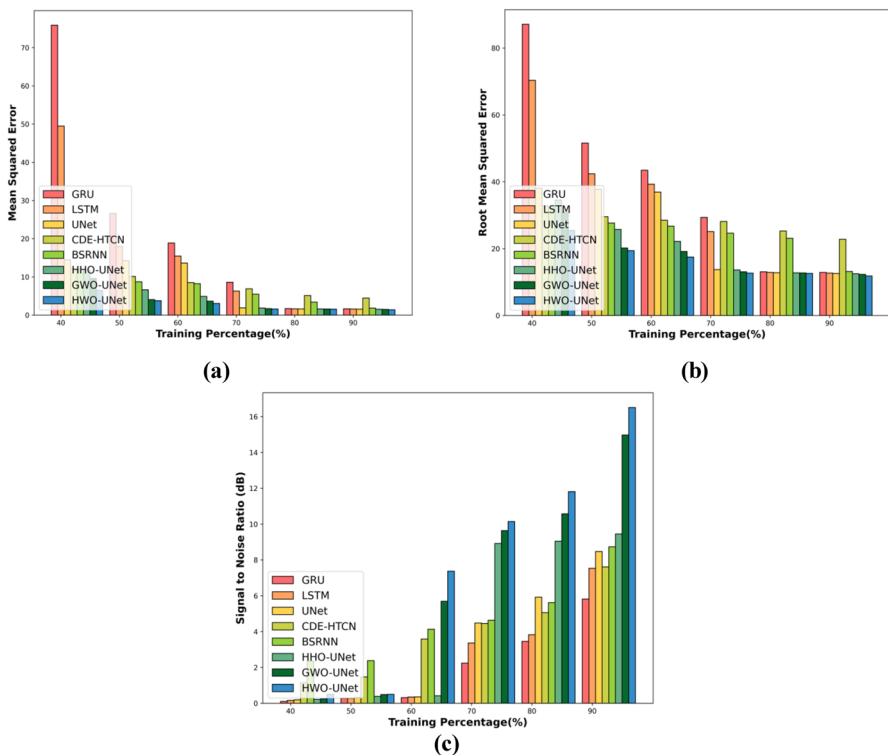


Fig. 9 Comparative evaluation for UrbanSound8K, **a** MSE, **b** RMSE, **c** SNR

48.70%, 53.89%, 47.10%, 42.74%, and 9.29 respectively. This investigation demonstrates that the HWO-UNet method performed better than the available methods.

4.6.2 Comparative Evaluation of U-Net Based on the HWO with Librispeech

In this comparative analysis, Librispeech is used as the reference method. The results obtained in terms of MSE are depicted in Fig. 10a. The MSE for HWO-UNet at TP 90 is 1.729, which shows an error difference of 0.30 over GRU. However, for LSTM, UNet, CDE-HTCN, BSRNN, HHO-UNet, and GWO-UNet, the error differences are 0.26, 0.22, 3.18, 2.00, 0.12, and 0.07 respectively. In Fig. 10b, the RMSE of the HWO-UNet method is 13.151 with an error difference of 1.09 over GRU, whereas the error differences over the techniques LSTM, UNet, CDE-HTCN, BSRNN, HHO-UNet, and GWO-UNet are 0.94, 0.82, 8.36, 7.67, 0.44, and 0.28 respectively. Moreover, the SNR of the HWO-UNet at TP 90 is 14.226 dB, which shows the performance improvement of 62.35% for GRU, 54.80% for LSTM and 34.46% for UNet, 49.80% for CDE-HTCN, 38.05% for BSRNN, 31.83% for HHO-UNet, and 7.20% for GWO-UNet, in Fig. 10c.

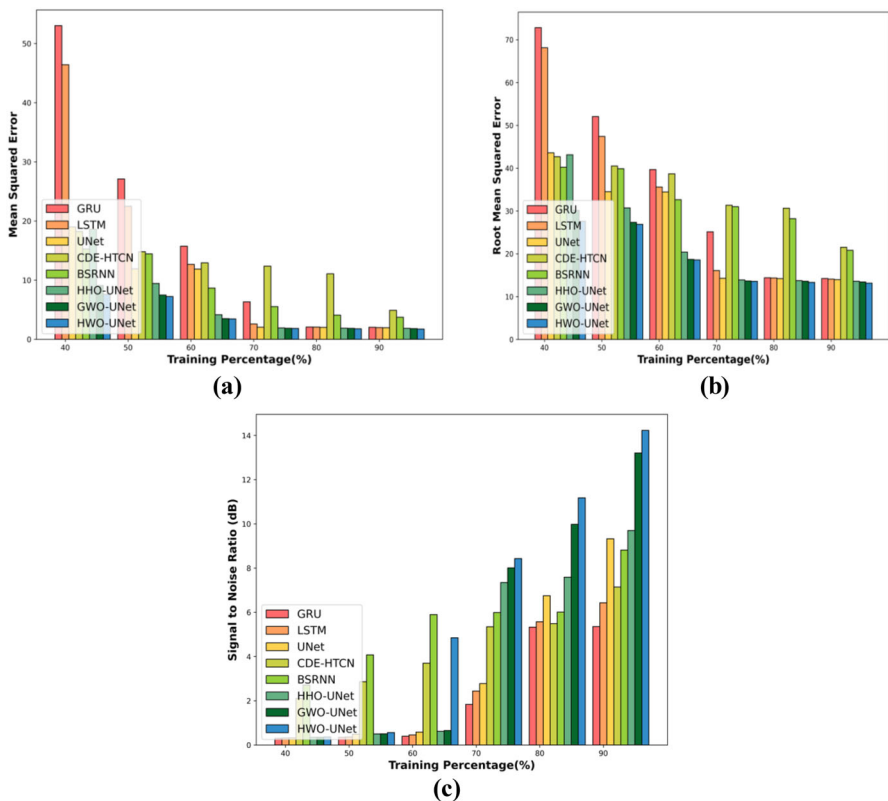


Fig. 10 Comparative evaluation for Librispeech, **a** MSE, **b** RMSE, and **c** SNR

4.6.3 Comparative Evaluation of U-Net Based on the HWO with MUSDB18

The performance comparison of the HWO-UNet model with other existing methods is revealed in Fig. 11. In Fig. 11a the MSE of the HWO-UNet model is 0.912, which is compared with certain methods and shows error differences of 1.08, 0.93, 0.83, 5.04, 1.30, 0.73, and 0.33 concerning methods, such as GRU, LSTM, UNet, CDE-HTCN, BSRNN, HHO-UNet, and GWO-UNet respectively, whereas in Fig. 11b the HWO-UNet attains the RMSE of 1.242, which shows the error differences of 2.63, 1.43, 1.42, 5.07, 3.45, 1.29, and 1.00 over the techniques GRU, LSTM, UNet, CDE-HTCN, BSRNN, HHO-UNet, and GWO-UNet respectively. The SNR of the HWO-UNet is obtained as 15.18 dB, and the performance improvement over the existing methods GRU, LSTM, UNet, CDE-HTCN, BSRNN, HHO-UNet, and GWO-UNet in comparison with HWO-UNet are 32.38%, 26.02%, 11.06%, 5.75%, 2.11%, 1.47%, and 0.11% respectively, which is depicted in Fig. 11c. Thus a more efficient performance is found in the HWO-UNet method than the other existing methods utilized such as GRU, LSTM, UNet, CDE-HTCN, and BSRNN.

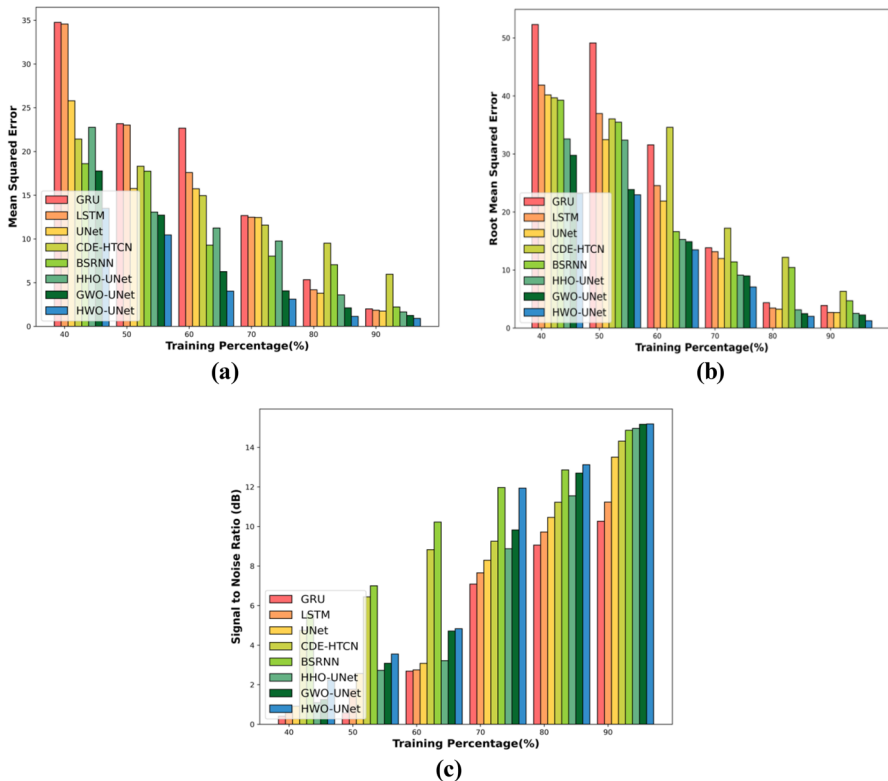


Fig. 11 Comparative analysis for MUSDB18, **a** MSE, **b** RMSE, **c** SNR

4.7 Comparative Discussion

Analysis of ASS systems is discussed in this section. GRU, LSTM, UNet, CDE-HTCN, BSRNN, HHO-UNet, and GWO-UNet are the techniques adopted for the comparative evaluation. Even while these current approaches are used to offer better solutions, they are encountered with limitations that reduce the model's effectiveness. Despite being computationally cheap, GRU is not very good at learning the long-term interdependence of complicated tasks with extensive datasets. The LSTM method's disadvantage is that it requires more processing power and is affected by gradient exploding and overfitting. The main limitation of the GWO-UNet is that it is unable to manage the enormous amount of datasets, and its escaping local solutions. The imbalanced nature of global exploration and local exploitation causes the HHO to enter the local optimum stagnation in the HHO-UNet model. However, the developed research, HWO-UNet offered as a solution to some of the shortcomings of the current approaches. When compared to existing methods, the UNet model always reconstructs the signal that minimizes distortion and achieves maximum performance with optimal solutions. Table 3 portrays the comparative discussion of the HWO-UNet and other competent techniques.

4.8 Statistical Analysis

The statistical analysis is accomplished for the HWO-UNet and the conventional techniques, in which the HWO-UNet obtained the low values of mean, min, and max for the evaluation metrics such as MSE and RMSE. Further, the HWO-UNet attained high values of mean, min, and max, for SNR on comparing with other competent techniques that declare the efficiency of the HWO-UNet for ASS. The statistical analysis with the Training percentage using the UrbanSound8K dataset is shown in Table 4.

4.9 Time Complexity Analysis

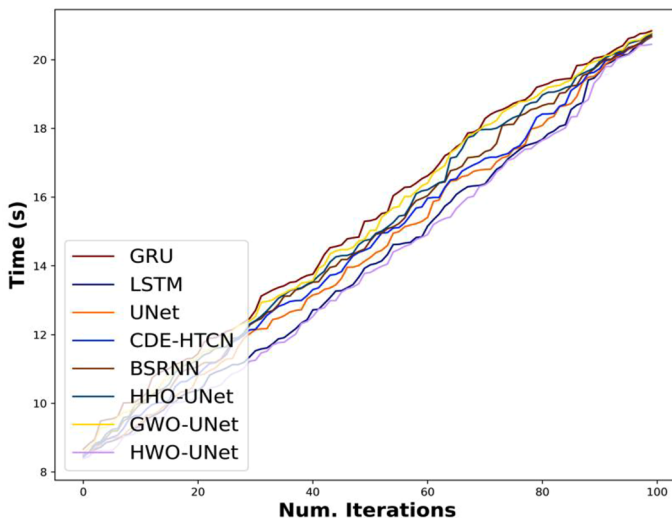
The computational analysis is conducted for the HWO-UNet model along with the competent techniques and is portrayed in Fig. 12. At iteration 20, the HWO-UNet model attained the computational time of 10.10 s, while the existing techniques such as GRU, LSTM, UNet, CDE-HTCN, BSRNN, HHO-UNet, and GWO-UNet attained the computational time of 11.31 s, 10.34 s, 10.46 s, 10.76 s, 11.02 s, 11.19 s, and 11.21 s respectively, which are higher compared to the HWO-UNet. Finally, at iteration 100, the HWO-UNet model attained the computational time of 20.45, while the existing techniques such as GRU, LSTM, UNet, CDE-HTCN, BSRNN, HHO-UNet, and GWO-UNet attained the computational time of 20.84 s, 20.66 s, 20.67 s, 20.70 s, 20.71 s, 20.76 s, and 20.78 s respectively. Additionally, the computational analysis demonstrated that the HWO-UNet performed better than other traditional methods. The strategy that is suggested makes use of HWO optimization, which speeds up the convergence and restricts the system from entering the local optimal solution. The HWO-UNet model's computation time is decreased by training U-net with the HWO method and attained the enhanced ASS.

Table 3 Comparative discussion based on Training percentage analysis

Methods/analysis	Training percentage								
	UrbanSound8K dataset			LibriSpeech dataset			MUSDB18 dataset		
	MSE	RMSE	SNR (dB)	MSE	RMSE	SNR (dB)	MSE	RMSE	SNR (dB)
GRU	1.661	12.88	5.82	2.027	14.23	5.36	1.990	3.871	10.26
LSTM	1.615	12.70	7.54	1.986	14.09	6.43	1.845	2.670	11.23
UNet	1.581	12.57	8.47	1.951	13.96	9.32	1.742	2.659	13.50
CDE-HTCN	4.490	22.78	7.61	4.909	21.50	7.14	5.954	6.311	14.30
BSRNN	1.826	13.16	8.73	3.733	20.81	8.81	2.212	4.693	14.86
HHO-UNet	1.560	12.49	9.45	1.847	13.59	9.70	1.644	2.530	14.95
GWO-UNet	1.515	12.31	14.97	1.803	13.42	13.20	1.240	2.239	15.16
Proposed HWO-UNet	1.402	11.84	16.51	1.729	13.15	14.23	0.912	1.242	15.18

Table 4 Statistical analysis

Methods/analysis	Training percentage								
	MSE			RMSE			SNR		
	Mean	Min	Max	Mean	Min	Max	Mean	Min	Max
GRU	22.23	1.66	75.90	39.58	12.89	87.12	2.04	0.11	5.82
LSTM	15.41	1.62	49.48	33.79	12.71	70.34	2.59	0.16	7.54
UNet	7.91	1.58	14.48	25.31	12.58	38.05	3.29	0.19	8.47
CDE-HTCN	7.88	4.49	12.14	27.83	22.79	32.73	3.89	1.14	7.61
BSRNN	6.60	1.83	11.94	24.58	13.17	32.22	4.64	2.35	8.73
HHO-UNet	4.75	1.56	11.92	20.23	12.49	34.53	4.74	0.21	9.45
GWO-UNet	3.70	1.52	9.59	18.08	12.31	30.97	6.94	0.24	14.97
Proposed HWO-UNet	2.99	1.40	6.49	16.58	11.84	25.47	7.81	0.50	16.51

**Fig. 12** Time complexity analysis

5 Conclusion

This research proposes a HWO-UNet method that works efficiently in ASS along with feature extraction using the spectrogram features, such as STFT and MFCC as well as the statistical features with added features, like audio fingerprinting and beat histogram. The HWO-UNet approach minimizes the overfitting issues and loss simultaneously. In addition, the developed approach also utilizes sampling technologies in the real-time dataset, allowing the system to function with large datasets. The performance of

the HWO-UNet method over other conventional techniques is evaluated in terms of RMSE and MSE as 1.242 and 0.912, respectively, whereas the SNR for the MUSDB18 dataset is 15.18 dB. Various existing techniques, including GRU, LSTM, U-Net, HHO-UNet, and GWO-UNet, have been compared with the HWO-UNet model, in which the HWO-UNet approach with skip connections grasp the pertinent features, provides optimal solutions, and maximizes the ASS performance. However, the utilization of numerous intricate convolutional computations in HWO-UNet results in a significant level of complexity in the developed model. Hence, the complexity of the developed HWO-UNet should be reduced to make it compact in the future. By adopting advanced attention mechanisms and algorithms, this research may be enhanced and separation accuracy can be further increased in the future.

Author Contribution All authors have made substantial contributions to conception and design, revising the manuscript, and the final approval of the version to be published. Also, all authors agreed to be accountable for all aspects of the work in ensuring that questions related to the accuracy or integrity of any part of the work are appropriately investigated and resolved.

Funding This research did not receive any specific funding.

Data Availability The data underlying this article are available in UrbanSound8k, Libri Speech, and MUSDB18. Libri Speech dataset: <https://www.tensorflow.org/datasets/catalog/librispeech>. UrbanSound8k: <https://urbansounddataset.weebly.com/urbansound8k.html>. MUSDB18: <https://sigsep.github.io/datasets/musdb.html>.

Declarations

Conflict of interest The authors declare no conflict of interest.

Ethical Approval Not applicable.

References

1. S.G. Abhyankar, S.S. Bharadwaj, G.S. Rani, P.G. Karigiri, S. Srikanth, and S. Gurugopinath, A survey on music genre classification using multimodal information processing and retrieval, in *2023 International Conference on Recent Trends in Electronics and Communication (ICRTEC)*, 1–6 (2023)
2. M. Altaf, T. Akram, M.A. Khan, M. Iqbal, M.M.I. Ch, C.H. Hsu, A new statistical features-based approach for bearing fault diagnosis using vibration signals. *Sensors* **22**(5), 2012 (2022)
3. R. Al-Wajih, S.J. Abdulkadir, N. Aziz, Q. Al-Tashi, N. Talpur, Hybrid Binary grey wolf with Harris Hawks optimizer for feature selection. *IEEE Access* **9**, 31662–31677 (2021)
4. Y. Bando, K. Sekiguchi, Y. Masuyama, A.A. Nugraha, M. Fontaine, K. Yoshii, Neural full-rank spatial covariance analysis for blind source separation. *IEEE Signal Process. Lett.* **28**, 1670–1674 (2021)
5. S. Chang, D. Lee, J. Park, H. Lim, K. Lee, K. Ko, and Y. Han, Neural audio fingerprint for high-specific audio retrieval based on contrastive learning, in *ICASSP 2021–2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 3025–3029 (2021)
6. M. Chatterjee, J. Le Roux, N. Ahuja, and A. Cherian, Visual scene graphs for audio source separation, in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 1204–1213 (2021)
7. J. Chen, C. Liu, J. Xie, J. An, N. Huang, Time-frequency mask-aware bidirectional LSTM a deep learning approach for underwater acoustic signal separation. *Sensors* **22**(15), 5598 (2022)
8. L. Chen, G. Chen, L. Huang, Y.S. Choy, W. Sun, Multiple sound source localization, separation, and reconstruction by microphone array A DNN-based approach. *Appl. Sci.* **12**(7), 3428 (2022)
9. Y.S. Chen, Z.J. Lin, M.R. Bai, A multichannel learning-based approach for sound source separation in reverberant environments. *EURASIP J. Audio Speech Music Process.* **2021**, 1–12 (2021)

10. P. Diel, A.J. Muñoz-Montoro, J.J. Carabias-Orti, and J. Ranilla, Efficient FPGA implementation for sound source separation using direction-informed multichannel non-negative matrix factorization. *J. Supercomput.* 1–23 (2024)
11. S. Gul, and M.S. Khan, A survey of audio enhancement algorithms for music, speech, bioacoustics, biomedical, industrial, and environmental sounds by image U-Net. *IEEE Acces* (2023).
12. R. Guo, Z. Luo, M. Li, A survey of optimization methods for independent vector analysis in audio source separation. *Sensors* **23**(1), 493 (2023)
13. N. Hassan, D.A. Ramli, Sparse component analysis (SCA) based on adaptive time-frequency thresholding for underdetermined blind source separation (UBSS). *Sensors* **23**(4), 2060 (2023)
14. T. T. Hasumi, Nakamura, N. Takamune, H. Saruwatari, D. Kitamura, Y. Takahashi, and K. Kondo, Empirical Bayesian independent deeply learned matrix analysis for multichannel audio source separation, in *2021 29th European Signal Processing Conference (EUSIPCO)* 331–335 (2021)
15. W.H. Heo, H. Kim, O.W. Kwon, Integrating dilated convolution into denseLSTM for audio source separation. *Appl. Sci.* **11**(2), 789 (2021)
16. Y. Hu, Y. Chen, W. Yang, L. He, H. Huang, Hierarchic temporal convolutional network with cross-domain encoder for music source separation. *IEEE Signal Process. Lett.* **29**, 1517–1521 (2022)
17. H. L. Huang, Lin, R. Tong, H. Hu, Q. Zhang, Y. Iwamoto, X. Han, Y.W. Chen, and J. Wu, Unet 3+: A full-scale connected UNet for medical image segmentation, in *ICASSP 2020–2020 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, 1055–1059 (2020).
18. R.R. Huizen, and F.T. Kurniati, Feature extraction with mel scale separation method on noise audio recordings. *arXiv preprint arXiv:2112.14930* (2021)
19. A.G. Hussien, L. Abualigah, R. Abu Zitar, F.A. Hashim, M. Amin, A. Saber, K.H. Almotairi, A.H. Gandomi, Recent advances in Harris Hawks optimization: A comparative study and applications. *Electronics* **11**(12), 1919 (2022)
20. R.J. Issa, Y.F. Al-Irhaym, Audio source separation using supervised deep neural network. *J. Phys. Conf. Ser.* **1879**(2), 022077 (2021)
21. N. Ito, R. Ikeshita, H. Sawada, T. Nakatani, A joint diagonalization-based efficient approach to underdetermined blind audio source separation using the multichannel Wiener filter. *IEEE/ACM Trans. Audio Speech Lang. Process.* **29**, 1950–1965 (2021)
22. V.S. Kadandale, J.F. Montesinos, G. Haro, and E. Gómez, Multi-channel u-net for music source separation, in *2020 IEEE 22nd international workshop on multimedia signal processing (MMSP)*, 1–6 (2020)
23. W.H. Lai, S.L. Wang, RPCA-DRNN technique for monaural singing voice separation. *EURASIP J. Audio Speech Music Process.* **2022**(1), 4 (2022)
24. C. Lan, J. Jiang, L. Zhang, and Z. Zeng, Blind source separation based on improved Wave-U-Net network. *IEEE Access* (2023)
25. B. Laufer-Goldshtein, R. Talmon, S. Gannot, Global and local simplex representations for multichannel source separation. *IEEE/ACM Trans. Audio Speech Lang. Process.* **28**, 914–928 (2020)
26. L. Li, H. Cai, H. Han, Q. Jiang, H. Ji, Adaptive short-time Fourier transform and synchrosqueezing transform for non-stationary signal separation. *Signal Process.* **166**, 107231 (2020)
27. Y. Li, X. Lin, J. Liu, An improved gray wolf optimization algorithm to solve engineering problems. *Sustainability* **13**(6), 3208 (2021)
28. Librispeechdataset: <https://www.openslr.org/12>. Accessed on June 2023.
29. Y. Luo, and J. Yu, Music source separation with band-split RNN. *IEEE/ACM Trans. Audio Speech Lang. Process.* (2023)
30. P. Magron, T. Virtanen, Online spectrogram inversion for low-latency audio source separation. *IEEE Signal Process. Lett.* **27**, 306–310 (2020)
31. S.G. Mali, M.V. Dhale, and S.P. Mahajan, Separation of multiple stationary sound sources using convolutional neural network, in *2021 6th International Conference for Convergence in Technology (I2CT)*, 1–6 (2021)
32. R.B. Mohite, and O.S. Lamba, Classifier comparison for blind source separation, in *2021 International Conference on Smart Generation Computing, Communication and Networking (SMART GENCON)* 1–5 (2021)
33. MUSDB18 dataset: <https://sigsep.github.io/datasets/musdb.html#musdb18-compressed-stems>. Accessed on June 2023.
34. W.K. Mutlag, S.K. Ali, Z.M. Aydam, B.H. Taher, Feature extraction methods: a review. *J. Phys. Conf. Ser.* **1591**(1), 012028 (2020)

35. T. Nakamura, S. Kozuka, H. Saruwatari, Time-domain audio source separation with neural networks based on multiresolution analysis. *IEEE/ACM Trans. Audio Speech Lang. Process.* **29**, 1687–1701 (2021)
36. T. Nakamura, and H. Saruwatari, Time-domain audio source separation based on Wave-U-Net combined with discrete wavelet transform, in *ICASSP 2020–2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* 386–390 (2020) IEEE.
37. V.W. Neo, C. Evers, and P.A. Naylor, Polynomial matrix eigenvalue decomposition-based source separation using informed spherical microphone arrays, in *2021 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 201–205 (2021)
38. M. Pezzoli, J.J. Carabias-Orti, M. Cobos, F. Antonacci, A. Sarti, Ray-space-based multichannel non-negative matrix factorization for audio source separation. *IEEE Signal Process. Lett.* **28**, 369–373 (2021)
39. J. Qian, X. Liu, Y. Yu, and W. Li, Stripe-transformer: deep stripe feature learning for music source separation. *EURASIP J. Audio Speech Music Process.* **2023**(1), 2(2023)
40. L.C. Reghunath, R. Rajan, Predominant audio source separation in polyphonic music. *EURASIP J. Audio Speech Music Process.* **2023**(1), 49 (2023)
41. S. Rouard, F. Massa, and A. Défossez, Hybrid transformers for music source separation, in *ICASSP 2023–2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* 1–5 (2023)
42. K. Saito, T. Nakamura, K. Yatabe, H. Saruwatari, Sampling-frequency-independent convolutional layer and its application to audio source separation. *IEEE/ACM Trans. Audio Speech Lang. Process.* **30**, 2928–2943 (2022)
43. K. Schulze-Forster, G. Richard, L. Kelley, C.S. Doire, R. Badeau, Unsupervised music source separation using differentiable parametric source models. *IEEE/ACM Trans. Audio Speech Lang. Process.* **31**, 1276–1289 (2023)
44. M. Schwabe, and M. Heizmann, Improved separation of polyphonic chamber music signals by integrating instrument activity labels. *IEEE Access* (2023)
45. T. Sgouros, A. Bousis, N. Mitianoudis, An efficient short-time discrete cosine transform and attentive multiresunet framework for music source separation. *IEEE Access* **10**, 119448–119459 (2022)
46. T. Sgouros, N. Mitianoudis, A novel directional framework for source counting and source separation in instantaneous underdetermined audio mixtures. *IEEE/ACM Trans. Audio Speech Lang. Process.* **28**, 2025–2035 (2020)
47. N. Siddique, S. Paheding, C.P. Elkin, V. Devabhaktuni, U-net and its variants for medical image segmentation: a review of theory and applications. *IEEE Access* **9**, 82031–82057 (2021)
48. O. Slizovskaia, G. Haro, E. Gómez, Conditioned source separation for musical instrument performances. *IEEE/ACM Trans. Audio Speech Lang. Process.* **29**, 2083–2095 (2021)
49. T.H. Tsai, P.Y. Liu, Y.H. Chiou, Hardware design for blind source separation using fast time-frequency mask technique. *Integration* **82**, 67–77 (2022)
50. Urbansound8k dataset: <https://urbansounddataset.weebly.com/urbansound8k.html>. Accessed on June 2023.
51. J.S. Wang, S. Guan, X.L.Z. Liu, Minimum-volume multichannel nonnegative matrix factorization for blind audio source separation. *IEEE/ACM Trans. Audio Speech Lang. Process.* **29**, 3089–3103 (2021)
52. J. Zhang, Music feature extraction and classification algorithm based on deep learning. *Sci. Program.* **2021**, 1–9 (2021)
53. L. Zhang, C.P. Lim, Y. Yu, M. Jiang, Sound classification using evolving ensemble models and Particle Swarm Optimization. *Appl. Soft Comput.* **116**, 108322 (2022)
54. M. Zhao, X. Yao, J. Wang, Y. Yan, X. Gao, Y. Fan, Single-channel blind source separation of spatial aliasing signal based on stacked-LSTM. *Sensors* **21**(14), 4844 (2021)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.